

LAB 06 ASSIGNMENT

Performance Comparison of DFS-Backtracking and Local Search Using Heuristics

Application: MAP COLORING PROBLEM

Understand the CSP → implement two solvers → measure time → compare performance

1. What is the purpose of this lab?

You will solve the same Map Coloring problem using TWO approaches.

Approach 1: DFS-Backtracking with MRV, LCV and Forward Checking.

Approach 2: Local Search using the Min-Conflicts heuristic.

You will increase the problem size and measure computation time.

Finally, you will decide which approach performs better as the problem becomes larger.

2. First understand the real-world problem

Map Coloring

Imagine a map divided into regions.

Each region must receive one color.

Rule: Two neighboring regions must NOT have the same color.

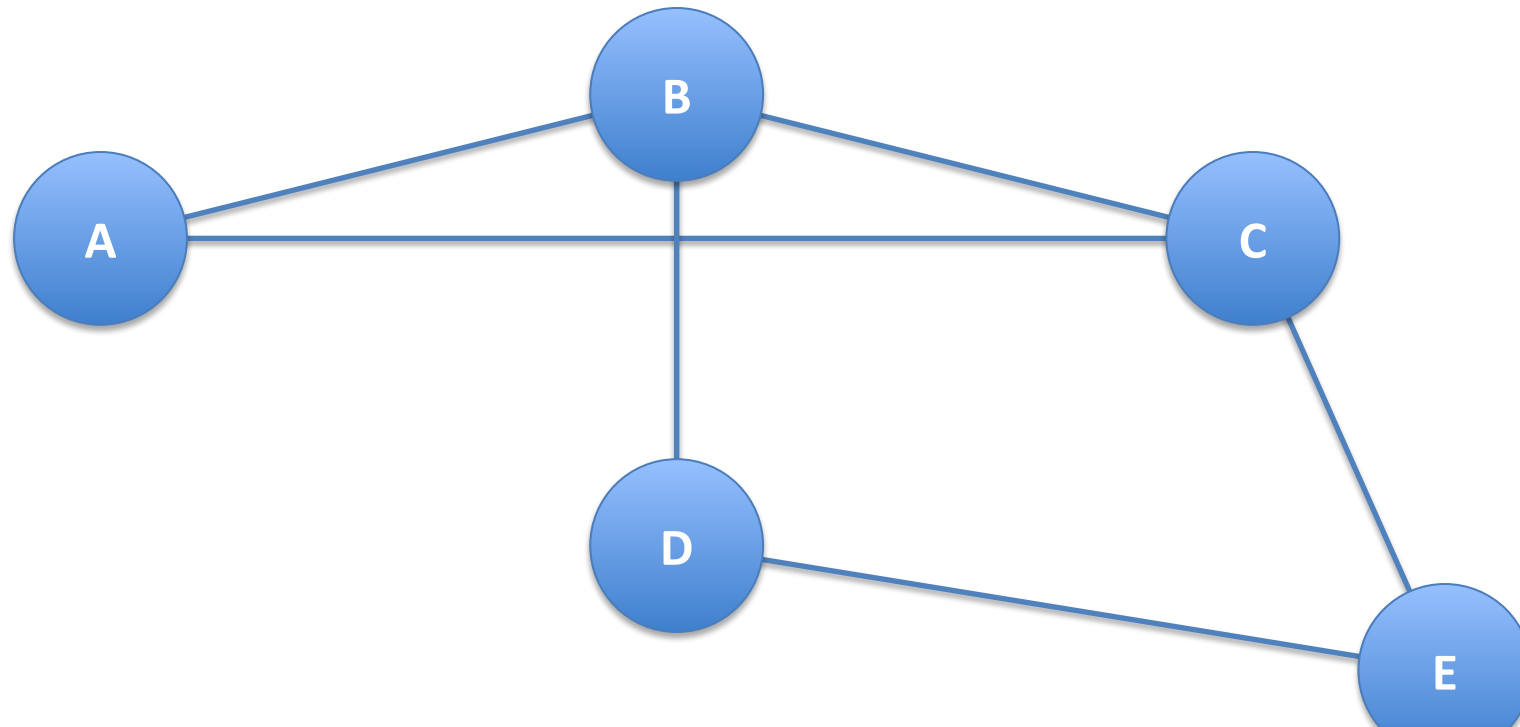
We will use exactly three colors:
Red, Green, Blue.

Why is this a CSP?

Because we have:

- Variables — regions
- Domain — available colors
- Constraints — neighboring regions must differ
- Goal — find a complete assignment that satisfies every constraint

3. Convert the map into a graph



Important mapping

Region = vertex (variable) | Neighbor relationship = edge (constraint)

4. The three parts of our CSP

VARIABLES

Each region is a variable.

Example:

A, B, C, D, E

Number of variables = number of regions.

DOMAIN

Every variable can choose one of:

{Red, Green, Blue}

Each region gets exactly ONE color.

CONSTRAINTS

For every edge (X,Y):

$X \neq Y$

So neighboring regions must have different colors.

Number of constraints = number of edges.

5. How do we count constraints?

ONE EDGE = ONE CONSTRAINT

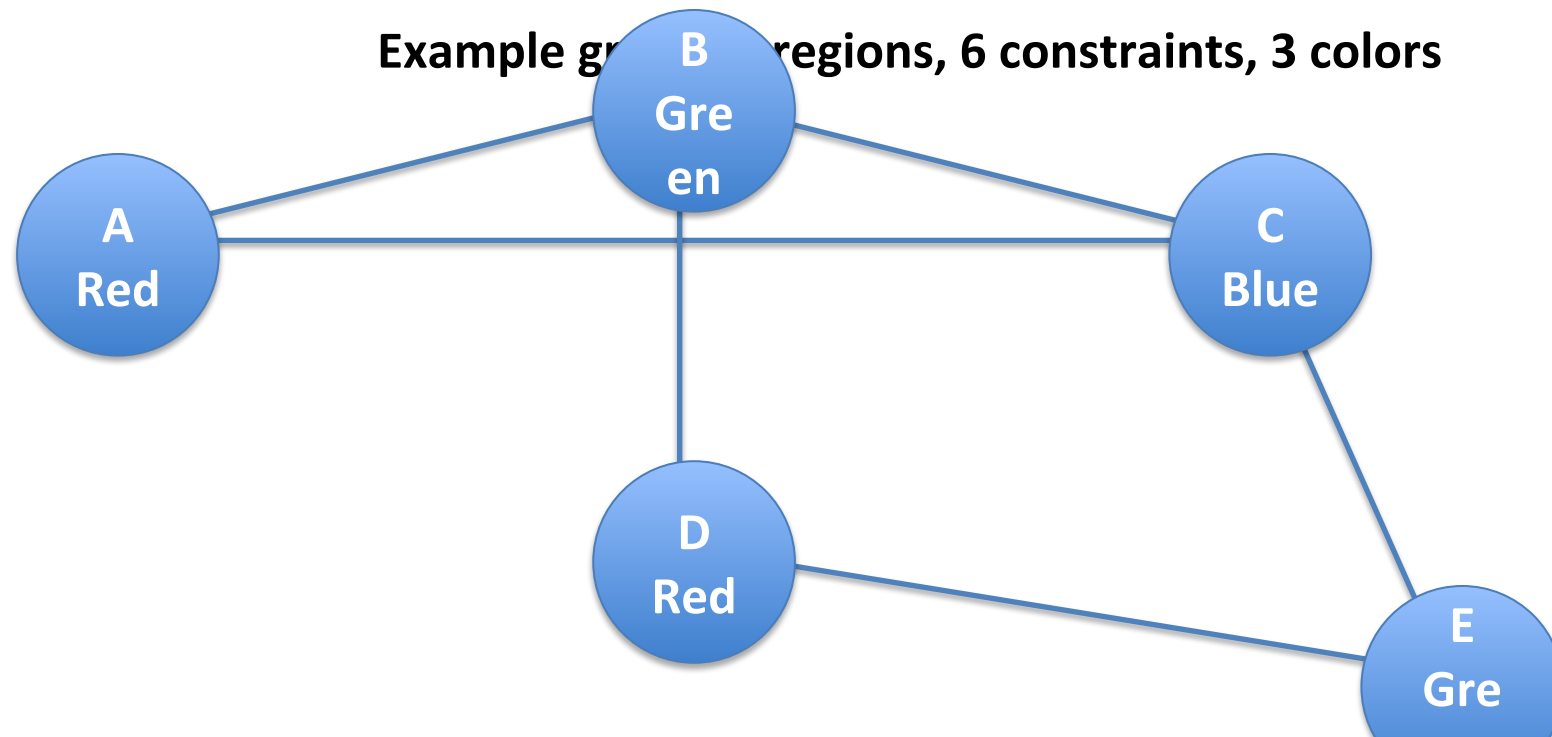
Edge	Constraint
A—B	$A \neq B$
A—C	$A \neq C$
B—C	$B \neq C$
B—D	$B \neq D$
C—E	$C \neq E$
D—E	$D \neq E$

This example has 6 edges.

Therefore, it has 6 constraints.

Do NOT count A—B and B—A separately; they represent the same constraint.

6. Worked example: find a valid coloring



Check every edge

A-B ✓ A-C ✓ B-C ✓ B-D ✓ C-E ✓ D-E ✓

No neighboring regions have the same color → VALID SOLUTION

7. What happens if a constraint is violated?

Invalid assignment

Suppose:

A = Red

B = Green

C = Blue

D = Red

E = Green

If A and D are connected, then:

A = D = Red

Constraint $A \neq D$ is violated $\rightarrow$ INVALID.

What must the algorithm do?

It must change an assignment.

DFS-Backtracking:

$\rightarrow$ go back and try another value.

Local Search:

$\rightarrow$ change a conflicted variable to a better value.

Both methods try to reach ZERO conflicts.

8. Algorithm 1 — DFS-Backtracking

DFS = Depth-First Search.

Assign a color to one region at a time.

After every assignment, check whether the constraints are still satisfied.

If the current choice causes a conflict, undo it and try another color.

Continue until all regions are assigned.

If no color works for a region, backtrack to an earlier decision.

Simple idea

Choose → Assign → Check → If failure, Backtrack → Continue

9. DFS heuristic 1 — MRV

MRV = Minimum Remaining Values

Choose the unassigned region with the FEWEST legal colors remaining.

Example:

A → {Red, Green}

B → {Red}

C → {Red, Green, Blue}

D → {Green, Blue}

B has only 1 choice → select B first.

Why MRV helps

Without MRV, we may make many unnecessary choices.

MRV says:

“Deal with the most restricted variable first.”

This often detects failure earlier and reduces the search tree.

10. DFS heuristic 2 — LCV

LCV = Least Constraining Value

When a variable has multiple colors, choose the color that removes the FEWEST options from its neighbors.

Example for region A:

Red removes 4 choices

Green removes 2 choices

Blue removes 1 choice

Choose Blue.

Why LCV helps

LCV keeps more possibilities available for the remaining regions.

Think:

“Choose the value that leaves the most freedom for others.”

11. DFS heuristic 3 — Forward Checking

What does it do?

After assigning a color to a region, remove that color from the domains of its unassigned neighbors.

Example:

A = Red

Then Red is removed from the domains of B and C if they are neighbors.

Why is it useful?

Suppose a neighbor's domain becomes:
{ }

That means it has NO legal color.

We know immediately that the current path cannot work → backtrack early.

12. DFS method — put the heuristics together

MRV → LCV → Assign → Forward Check → Backtrack if needed

1	MRV Choose the most restricted region.
2	LCV Choose the least constraining color.
3	Assign Give the selected color to the
4	Forward Check Remove that color from neighbor
5	Backtrack If a domain becomes empty, undo and try another choice.

What to record for DFS

- Solution found or not
- Execution time
- Number of assignments attempted
- Number of backtracks
- Final coloring
- Number of variables and constraints

13. Algorithm 2 — Local Search with Min-Conflicts

Start with a COMPLETE coloring, even if some constraints are violated.

Find regions that are currently involved in conflicts.

Choose one conflicted region.

Try the available colors for that region.

Choose the color that produces the FEWEST conflicts.

Repeat until there are ZERO conflicts or 10,000 iterations are reached.

Main idea

Do not restart from the beginning — keep improving the current solution.

14. Worked example — Min-Conflicts

Current coloring

A = Red
B = Red ← conflict with A
C = Blue
D = Green
E = Red

Choose B because it is involved in a conflict.

Color for B	Conflicts
Red	1
Green	0
Blue	1

Decision

Green gives the minimum number of conflicts: 0.

So change B → Green.

15. Important difference between the two methods

DFS-Backtracking

Systematic search

Starts with an empty assignment.

Explores choices in a search tree.

Can backtrack when a choice fails.

With correct CSP search, it can find a solution whenever one exists.

Local Search / Min-Conflicts

Starts with a complete assignment.

Improves the assignment step by step.

Focuses on conflicts rather than building a tree.

Not guaranteed to find a solution within the iteration limit.

16. Experimental setup — increase the problem size

Test	Variables / Regions
1	3
2	5
3	7
4	10
5	15
6	20
7	30
8	40
9	50

Rules for every test case

- Use exactly 3 colors: Red, Green, Blue.
- Generate a graph with the selected number of regions.
- Count its edges = number of constraints.
- Ensure the graph is 3-colorable.
- Use the SAME graph for DFS and Local Search.
- Do not change the problem between the two algorithms.

17. Do not confuse these three quantities

Problem size

Number of variables = number of regions.

Example:
50 regions → 50 variables.

This changes from one test case to another.

Constraints

Number of edges in the graph.

Example:
50 regions and 120 edges →
120 constraints.

This depends on the generated graph.

10 runs

Run EACH algorithm 10 times for the SAME test case.

Average the 10 execution times.

10 runs $\neq$ 10 variables.

Local Search iterations are separate from these 10 runs.

18. How to measure computation time

For each test case

1. Generate one graph.
2. Run DFS 10 times on that graph.
3. Run Local Search 10 times on that graph.
4. Record each execution time.
5. Calculate the average.
6. Report the actual result.

Average time

Average Time =

$$(\text{Time}_1 + \text{Time}_2 + \dots + \text{Time}_{10}) \div 10$$

Use `time.perf_counter()` in Python.

Report the result in milliseconds (ms).

Do NOT invent or assume time values.

19. Required results table

Variables	Constraints	DFS Avg. Time (ms)	Local Search Avg. Time (ms)	Backtracks	Iterations
3	actual	actual	actual	actual	actual
5	actual	actual	actual	actual	actual
7	actual	actual	actual	actual	actual
10	actual	actual	actual	actual	actual
15	actual	actual	actual	actual	actual
20	actual	actual	actual	actual	actual
30	actual	actual	actual	actual	actual
40	actual	actual	actual	actual	actual
50	actual	actual	actual	actual	actual

Fill the table using values produced by your program.

20. Graphs you must generate

Graph 1 — Variables vs Time

X-axis:

Number of Variables (Regions)

Y-axis:

Average Computation Time (ms)

Plot TWO curves:

- DFS-Backtracking
- Local Search

Graph 2 — Constraints vs Time

X-axis:

Number of Constraints

Y-axis:

Average Computation Time (ms)

Plot TWO curves:

- DFS-Backtracking
- Local Search

21. What should you analyze?

1. Which method is faster for small problems?
2. What happens when the number of regions increases?
3. What happens when the number of constraints increases?
4. Does DFS produce more backtracks for larger problems?
5. Does Min-Conflicts require more iterations for harder problems?
6. How do MRV, LCV and Forward Checking improve DFS?
7. Is Local Search guaranteed to find a solution within 10,000 iterations?
8. At approximately what problem size does one method outperform the other?

22. What exactly do you have to implement?

CSP / Graph

- Graph representation
- Variables / regions
- Domain = 3 colors
- Constraint checking
- Edge counting
- 3-colorable graph generation

DFS Solver

- DFS-Backtracking
- MRV
- LCV
- Forward Checking
- Assignment count
- Backtrack count
- Timing

Local Search

- Random complete coloring
- Conflict detection
- Min-Conflicts
- Max 10,000 iterations
- Iteration count
- Conflict count
- Timing

23. Important rules — read before coding

Use exactly the same graph instance for both algorithms in each test case.

Use exactly 3 colors for every test case.

Count every edge only once; number of edges = number of constraints.

Ensure generated graphs are 3-colorable.

Run each algorithm 10 times for every test case.

Use actual execution times from your program.

Use `time.perf_counter()` for timing.

Stop Local Search after 10,000 iterations if no solution is found.

Record whether each algorithm actually found a solution.

24. Final picture — what are we comparing?

SAME CSP

DFS-Backtracking

MRV



LCV



Assign



Forward Checking



Backtrack when necessary

Local Search

Random complete coloring



Find conflict



Min-Conflicts



Change color



Repeat

Compare

Variables

Constraints

Execution Time

Backtracks

Iterations

Solution Found

Goal: understand how search strategy and heuristics affect performance.

25. Learning outcomes

Formulate a real problem as a Constraint Satisfaction Problem.

Identify variables, domains and constraints.

Represent Map Coloring using a graph.

Implement DFS-Backtracking with MRV, LCV and Forward Checking.

Implement Local Search using Min-Conflicts.

Measure and average computation time.

Study the effect of increasing variables and constraints.

Interpret tables, graphs and algorithmic performance.

ONE-MINUTE SUMMARY

CSP Element	Meaning
Variables	Regions
Domain	{Red, Green, Blue}
Constraints	Edges between neighboring regions
Rule	Connected regions must have different colors
DFS	MRV + LCV + Forward Checking + Backtracking
Local Search	Min-Conflicts
Experiments	3, 5, 7, 10, 15, 20, 30, 40, 50 regions
Runs	10 runs per algorithm per test case
Graphs	Variables vs Time AND Constraints vs Time

Build it → Run it → Measure it → Plot it → Explain it